U01 · Introducción a la programación

Antes de escribir una sola línea de Java vamos a entender **qué es realmente programar**. Esta unidad es la base de todo el módulo: si la dominas, el resto del curso te costará mucho menos.

1. ¿Qué es un programa?

La RAE define un programa como un *"conjunto de instrucciones que permite a un ordenador realizar funciones diversas"*. Dicho de otra forma más útil: un programa es una **receta** que un ordenador sigue al pie de la letra, sin criterio propio — hace exactamente lo que le dices, ni más ni menos (y ese es justo el motivo por el que los errores de programación existen).

Tres términos que se confunden mucho y que conviene distinguir desde el minuto uno:

- **Programa**: un conjunto de instrucciones ejecutables por un dispositivo.
- **Proceso**: un programa *en ejecución*, en un instante concreto (con su propio estado: qué instrucción está ejecutando, qué valores tienen sus variables, qué ficheros tiene abiertos…).
- **Aplicación**: uno o más programas junto con su documentación y los ficheros que necesita para funcionar — el conjunto que el usuario final percibe como "una herramienta".

> **La caja negra**
>
> Una forma útil de pensar en cualquier programa, sin importar lo complejo que sea por dentro: imagina que es una **caja negra**. No te importa cómo funciona por dentro, solo te importa que **lee** información del exterior (teclado, fichero, red, sensor…), la **transforma** y **escribe** un resultado (pantalla, fichero, red…). Vamos a pasarnos el curso abriendo esa caja negra, pieza a pieza.

2. ¿Qué es un algoritmo?

Un programa no es más que la traducción, a un lenguaje que entiende el ordenador, de un **algoritmo**: una secuencia finita y ordenada de pasos, sin ambigüedad, que resuelve un problema. Una receta de cocina es el ejemplo clásico — tiene unos ingredientes (entrada), unos pasos ordenados (proceso) y un plato final (salida):

> **Freír un huevo** — Entrada: huevo, aceite, sartén, fuego. Salida: huevo frito.
>
> 1. Pon aceite en la sartén.
> 2. Pon la sartén al fuego.
> 3. Cuando el aceite esté caliente, casca el huevo dentro.
> 4. Cubre el huevo de aceite.
> 5. Cuando esté hecho, retíralo.

Para que algo merezca llamarse algoritmo, tiene que cumplir unas condiciones **obligatorias**:

- **Resuelve el problema** para el que fue pensado — si no, no vale.
- **Es independiente de la plataforma**: no depende de ningún sistema operativo, lenguaje o hardware concretos (eso ya es cosa del programa que lo implementa).
- **Es preciso**: sus resultados tienen la exactitud que hace falta.
- **Es finito**: acaba siempre, en algún momento. Un algoritmo que en algún caso no termina nunca no es válido.
- **Es repetible**: puede ejecutarse una y otra vez con el mismo resultado para las mismas entradas.

Y hay otras cualidades **deseables**, que no todo algoritmo tiene pero conviene perseguir: que sea **válido** (sin errores), **eficiente** (resuelve el problema con pocos pasos y poca memoria) y, en el mejor de los casos, **óptimo** (el más eficiente posible, sin errores — el santo grial de cualquier programador).

3. Una historia muy breve de los lenguajes de programación

No hace falta memorizar fechas, pero entender *por qué* existen tantos lenguajes distintos te va a ayudar a entender mejor por qué Java es como es.

De los cables a las palabras

- **Código máquina (1ª generación)**: las primeras máquinas de propósito único (como el COLOSSUS de la Segunda Guerra Mundial) se "programaban" recableando circuitos. Cuando llegó la idea de guardar las instrucciones en memoria (ENIAC), programar pasó a ser escribir directamente combinaciones de ceros y unos: el **código máquina**. Es el único lenguaje que un procesador entiende de verdad, y sigue siendo distinto para cada tipo de procesador — nada portable, y prácticamente imposible de escribir a mano hoy en día.
- **Ensamblador (2ª generación)**: cambia cada combinación de bits por una palabra nemotécnica (`SUM`, `MOV`, `ADD`…) con traducción directa a código máquina. Un programa llamado *ensamblador* hace esa traducción. Sigue sin ser portable (cada procesador tiene el suyo), pero al menos ya se puede leer.
- **Lenguajes de alto nivel (3ª generación)**: el código ya no se parece en nada al hardware — lo traduce un **compilador** o un **intérprete**. En 1957 aparece **FORTRAN**, considerado el primer lenguaje de alto nivel. Le siguen **LISP** (1958, cálculo simbólico), **COBOL** (1960, gestión), **BASIC** (1964, pensado para aprender), **C** (1972, Dennis Ritchie) y **Pascal** (con fines didácticos, de Niklaus Wirth). Todos ellos sientan las bases de cómo programamos hoy.
- **Lenguajes de 4ª generación**: suben aún más el nivel — le dices al lenguaje *qué* quieres, no *cómo* conseguirlo. El ejemplo más conocido es **SQL**.
- **Lenguajes orientados a objetos**: desde **Simula** (1964, el primero con la idea de "objeto") hasta **C++**, y de ahí a **Java** (1995) y **C#** (2000, con ideas tomadas de C++ y del propio Java).

El mismo “Hola, mundo” a través de la historia

Compara cómo cambia la forma de decir lo mismo, según el lenguaje y la época:

> **Ensamblador (Intel 8086)**

> **FORTRAN (1957)**

> **COBOL (1960)**

> **C (1972)**

> **Pascal**

> **Java**

> **Python**

Fíjate en la tendencia: cada generación necesita **menos líneas** y se parece **más al español/inglés** para decir exactamente lo mismo.

Cómo se clasifican los lenguajes hoy

- **Nivel de abstracción**: lenguajes de **bajo nivel** (ensamblador, muy cercano al hardware) frente a lenguajes de **alto nivel** (Java, Python… mucho más cercanos a cómo pensamos los humanos).
- **Tipado**: lenguajes **fuertemente tipados** (como Java, donde cada variable tiene un tipo fijo que el compilador vigila) frente a lenguajes de **tipado flexible** (como Python o JavaScript).
- **Paradigma**: la “filosofía” con la que se organiza el código — lo vemos en detalle en el siguiente apartado.

4. Paradigmas de programación

Un **paradigma** es el estilo o la filosofía con la que se organiza un programa. No hay uno “mejor” que otro — cada uno encaja mejor en un tipo de problema, y muchos lenguajes modernos (Java incluido) combinan varios.

Paradigma imperativo

El programador describe, paso a paso, **cómo** resolver el problema — como si diera órdenes concretas, una tras otra.

- **Desestructurada**: predomina el instinto del programador sobre cualquier método, con abuso de saltos incondicionales (`GOTO`). Cuanto más grande el programa, más difícil de seguir.

```
10 SUMA = 0
20 PRINT "Número: "
30 INPUT N
40 IF N < 0 THEN GOTO 70
50 SUMA = SUMA + N
60 GOTO 20
70 PRINT "La suma es " SUMA
```

- **Estructurada**: solo se permiten tres tipos de sentencias — **secuenciales**, **alternativas** (si/sino) e **iterativas** (bucles) — sin saltos incondicionales. El mismo programa de antes, ya legible:

```
PROGRAM sumador;
VAR suma, n: INTEGER;
BEGIN
  suma := 0;
  REPEAT
    WRITE('Número: ');
    READ(n);
    IF n >= 0 THEN suma := suma + n;
  UNTIL n < 0;
  WRITE('La suma es ', suma);
END.
```

- **Modular**: además de estructurar, se organiza el código en **módulos independientes** (subprogramas), cada uno con su propia tarea. Es la base de toda la programación moderna, y la retomamos a fondo en el apartado 8.

Paradigma orientado a objetos

En vez de datos y sentencias sueltas, el programa se organiza en **objetos**: cada uno agrupa sus propios datos y el código que sabe manipularlos, y se parece más a cómo pensamos en el mundo real (una persona, una factura, un coche…).

```
public class Ejemplo {
    public static void main(String[] args) {
        Saludo s1 = new Saludo("Hola");
        s1.saluda("María");
    }
}

class Saludo {
    private String saludo;
    public Saludo(String frase) { saludo = frase; }
    public void saluda(String nombre) {
        System.out.println(saludo + ", " + nombre);
    }
}
```

Es el paradigma **principal de Java**, y en el que vas a vivir la mayor parte de este curso (unidades 4 a 7).

Paradigma declarativo

Aquí no describes *cómo* resolver el problema, sino *qué* resultado quieres — es el sistema el que decide cómo conseguirlo.

- **Funcional** (LISP, y cada vez más presente en Java a través de la API Stream — unidad 6): no distingue entre código y datos; se programa combinando funciones.
- **Lógico** (PROLOG): usa reglas y lógica formal para deducir resultados.
- El ejemplo declarativo más habitual en tu día a día como programador será **SQL** (unidades 8-9): le pides *qué* datos quieres, no *cómo* recorrer la base de datos para encontrarlos.

5. Cómo se ejecuta un programa: compiladores, intérpretes y máquinas virtuales

El código que escribes (**código fuente**) es solo texto. Para que la CPU lo entienda hace falta traducirlo a código máquina, y hay dos caminos:

Intérpretes → lenguajes interpretados. Traducen y ejecutan cada instrucción, una a una, en el momento.

- ✅ Resultados inmediatos, ideal para prototipos; si las 3 primeras líneas son correctas, se ejecutan aunque la 4ª tenga un error.
- ❌ Código menos optimizado y más lento; necesitas tener el intérprete instalado para ejecutar nada; los errores de las ramas que no se ejecutan quedan escondidos hasta que sí se ejecutan.

Compiladores → lenguajes compilados. Traducen *todo* el código fuente de una vez, revisándolo entero (análisis léxico, sintáctico y semántico) antes de generar un ejecutable.

- ✅ Detectan errores antes de ejecutar nada; el código generado va más optimizado y rápido; fuerzan buenos hábitos.
- ❌ Compilar lleva tiempo; el ejecutable resultante solo sirve para la plataforma para la que se compiló.

> ⓘ **¿Y Java qué hace?**
>
> Java hace un poco de cada cosa, y es la razón de su famoso lema *"write once, run anywhere"*. Tu código `.java` se **compila** a un formato intermedio llamado **bytecode** (ficheros `.class`), que no es código máquina real sino instrucciones para una máquina virtual. Después, la **JVM (Java Virtual Machine)** —que sí es específica de cada sistema operativo— **interpreta** (o compila en caliente, con el JIT) ese bytecode. Por eso el mismo `.class` funciona igual en Windows, Linux o Mac: solo necesitas tener instalada la JVM correspondiente. Los `.jar` son el formato en el que se empaquetan y distribuyen esos bytecodes ya listos para ejecutar.

6. Las herramientas de un programador

Para pasar de una idea a un programa funcionando necesitas varias piezas de software:

- **Editor de código**: donde escribes, con autocompletado, resaltado de sintaxis y marcado de errores.

- **Compilador / intérprete**: traduce tu código, como acabamos de ver.
- **Depurador (debugger)**: te deja ejecutar el programa paso a paso, vigilando el valor de las variables en cada momento — tu mejor amigo para cazar errores.
- **IDE (Integrated Development Environment)**: un único programa que junta todo lo anterior (y mucho más: gestión de proyectos, control de versiones, conexión a bases de datos…).

> **Nuestro IDE: IntelliJ IDEA**
>
> En este curso usamos **IntelliJ IDEA** como IDE oficial. Todo lo que veas de capturas, menús, atajos de teclado y depuración estará basado en él — es uno de los IDEs profesionales más usados del mundo para Java, así que además de aprender a programar, estarás aprendiendo una herramienta que se usa de verdad en la industria.

7. ¿Qué tipo de programas vamos a construir?

Los programas se pueden clasificar, a grandes rasgos, en cuatro formatos:

Tipo	Qué es	Ejemplo
Consola	Interactúa por texto, sin ventanas	Los primeros programas que harás en este curso
GUI (interfaz gráfica)	Ventanas, botones, menús	Swing/AWT — unidad 9
Servicios / *daemons*	Se ejecutan en segundo plano, sin interacción directa	Un proceso que vigila una carpeta y procesa ficheros nuevos
Web	Se ejecuta en un servidor y se accede desde el navegador	Fuera del alcance de este módulo, pero es el destino de muchos programas Java reales

Empezamos por consola porque es el formato más simple para centrarnos en la lógica sin distraernos todavía con ventanas ni clics — a partir de ahí, iremos subiendo la complejidad unidad a unidad.

8. El ciclo de vida de un programa

Escribir código es solo una fase del proceso. Un desarrollo de software serio se parece más a **encargar una tarta de bodas** que a fabricar un producto en cadena:

1. **Análisis**: hablas con quien quiere el software y le preguntas — "¿de qué sabor?", "¿para cuánta gente?" — hasta entender exactamente qué necesita. Es la fase que más se salta la gente sin experiencia, y la que más caro sale saltarse.
2. **Diseño**: decides *cómo* se va a resolver el problema antes de escribir código — qué pantallas, qué estructuras de datos, qué tecnologías. Es dibujar el plano antes de construir.
3. **Implementación (codificación)**: aquí, por fin, mezclas los ingredientes y los metes al horno — escribes el programa en un lenguaje concreto.
4. **Pruebas**: pruebas un trozo de tarta antes de servirla. Compruebas que el programa hace lo que se esperaba, con casos normales y casos límite.
5. **Despliegue y mantenimiento**: entregas la tarta… y sigues ahí por si hay que ajustar algo. Es, con diferencia, la fase que más dura en el tiempo.

⭐ **Be the Code**: la mayoría del código que vas a mantener en tu vida profesional **no lo habrás escrito tú**. Documentar bien tus programas (comentarios claros, nombres de variable que se entiendan) no es un capricho estético: es pensar en la persona (a veces tú mismo, seis meses después) que va a tener que entender tu código sin preguntarte.

La **documentación** de un proyecto se divide en dos tipos: la **interna** (los propios comentarios dentro del código fuente) y la **externa** (manuales de usuario, de mantenimiento, documentos de diseño…). Ambas son parte del trabajo, no un extra opcional.

9. Elementos de un programa

Todo lo que un programa manipula tiene tres atributos: un **nombre** (cómo lo identificas), un **tipo** (qué valores puede tomar) y un **valor** (el dato concreto que tiene en cada momento).

- **Constante**: su valor no cambia durante la ejecución. Es muy útil para no repetir "números mágicos" por todo el código — si en vez de escribir `0.21` en cada cálculo de IVA le das nombre a una constante `IVA`, el día que cambie el tipo solo tocas un sitio.
- **Variable**: su valor sí puede cambiar mientras el programa se ejecuta.

- **Expresión**: una combinación de datos y operadores que produce un resultado. Según ese resultado, hablamos de expresiones **numéricas** (`pi * radio * radio`), **alfanuméricas/de texto** (`"Don " + "José"`) o **booleanas/lógicas** (`(a > 1) Y (b < 5)`).

Operadores y su orden de evaluación

Tipo	Operadores
Relacionales	`>` `<` `>=` `<=` `==` `<>` (distinto)
Aritméticos	`+` `-` `*` `/` `mod` (resto) `^` (potencia)
Lógicos	`Y`/`AND` · `O`/`OR` · `NO`/`NOT`

Cuando una expresión mezcla varios operadores, no se evalúan de izquierda a derecha sin más — hay un **orden de precedencia**:

1. Paréntesis `()`
2. Potencia `^`
3. Multiplicación y división `* / mod`
4. Suma y resta `+ -`
5. Comparaciones relacionales `< <= > >=`
6. Negación `NOT`
7. Conjunción `AND`
8. Disyunción `OR`

> ⚠ **¡Cuidado con la precedencia!**
>
> `9 + 6 / 3` da **11**, no 5 — la división se calcula antes que la suma. Si de verdad quieres sumar primero, tienes que forzarlo con paréntesis: `(9 + 6) / 3` sí da 5. Usa paréntesis siempre que tengas la más mínima duda: cuestan una tecla y evitan errores muy difíciles de encontrar.

10. Algoritmos y pseudocódigo

Antes de traducir un algoritmo a Java (unidad 2), vamos a aprender a expresarlo en **pseudocódigo**: una notación intermedia, en español y sin la sintaxis estricta de ningún lenguaje real, que nos deja pensar en la lógica sin pelearnos todavía con punto y coma ni llaves. No existe un pseudocódigo "oficial" — usaremos una variante compatible con PSeInt, un intérprete de pseudocódigo gratuito que te permite ejecutar y comprobar tus algoritmos antes incluso de instalar IntelliJ.

Declarar y asignar datos

```
Definir sueldo, PI Como Real;
Definir nombre Como Cadena;
Definir edad Como Entero;
Definir casado Como Logico;

PI <- 3.14;
nombre <- "Jose";   // el texto va entre comillas
Leer sueldo;         // es buena idea avisar antes de leer
casado <- Verdadero;
```

`identificador <- valor` significa "copia `valor` en `identificador`" — no es una igualdad matemática, es una **asignación**, y el orden importa muchísimo. Si `valor` es una expresión, primero se calcula y luego se guarda el resultado.

Entrada y salida

```
Leer x;              // lee un dato del exterior y lo guarda en x
Escribir x, y;        // muestra datos por pantalla
```

Buenas costumbres al escribir pseudocódigo

- **Indenta**: si unas instrucciones están dentro de otras (dentro de un `Si` o un bucle), dejar sangría a la izquierda no es estético, es lo que te permite *ver* la lógica del programa de un vistazo.
- **Comenta** con `//` lo que no sea obvio con solo leer el código.
- Prefiere minúsculas y nombres que se entiendan (`sueldo`, no `s` o `x7`).

11. Expresiones lógicas

Todas las instrucciones de control de flujo (decidir, repetir) se apoyan en **expresiones lógicas**: fórmulas que siempre dan `Verdadero` o `Falso`, normalmente comparando datos (`n1 > 8`) o combinando comparaciones con `Y`, `O` y `NO`.

> **Un caso que engaña: la hora de tutoría**
>
> "La hora de tutoría es los lunes y martes a las 16 horas." ¿Cuál de estas expresiones es correcta?
>
> ```
> a) dia = "lunes" O dia = "martes" Y hora == 16
> b) (dia = "lunes" O dia = "martes") Y hora == 16
> ```
>
> Parece que deberían significar lo mismo, pero no: como `Y` tiene más precedencia que `O` (igual que la multiplicación tiene más precedencia que la suma), la opción **a)** se interpreta como *"es lunes, o es martes a las 16h"* — ¡tutoría todo el lunes! La que de verdad expresa "lunes o martes, y además a las 16h" es la **b)**, con paréntesis explícitos. Ante la duda, paréntesis siempre.

12. Sentencias alternativas (decidir)

Simple — solo actúa si la condición es verdadera:

```
Si nota >= 5 Entonces
    aprobado <- Verdadero;
FinSi
```

Doble — una rama para el caso verdadero y otra para el falso:

```
Si x <> 0 Entonces
    Escribir "Inverso de ", x, " = ", 1/x;
Sino
    Escribir "No tiene inverso";
FinSi
```

Anidadas — un `Si` dentro de otro, para más de dos posibilidades:

```
Si nota >= 5 Entonces
    Si nota >= 7 Entonces
        Si nota >= 9 Entonces
            Escribir "Sobresaliente";
        Sino
            Escribir "Notable";
        FinSi
    Sino
        Escribir "Aprobado";
    FinSi
Sino
    Escribir "Suspenso";
FinSi
```

Múltiples — cuando anidar se vuelve incómodo, `Según...Hacer` compara una expresión contra varios valores posibles (el equivalente en pseudocódigo del `switch` que verás en la unidad 3):

```
Leer opcion;
Según opcion Hacer
    1: Leer a, b;
    2: Escribir "Suma: ", a + b;
    3: Escribir "Resta: ", a - b;
    De Otro Modo: Escribir "Opción incorrecta";
FinSegun
```

13. Sentencias iterativas (repetir)

¿Por qué necesitamos bucles? Prueba a escribir, sin repetir, un algoritmo que muestre los números del 1 al 1000 — o uno que lea números hasta que el usuario meta un 10. Sin una forma de repetir instrucciones, sencillamente no se puede.

Mientras-Hacer: comprueba la condición *antes* de cada vuelta — si es falsa desde el principio, el bucle no se ejecuta ni una vez.

```
i <- 1;
Mientras i <= 10 Hacer
    Escribir i;
    i <- i + 1;
FinMientras
```

Repetir-Hasta: comprueba la condición *después* — el bucle se ejecuta siempre al menos una vez, y termina **cuando la condición se cumple** (al revés que `Mientras`).

```
x <- 1;
Repetir
    Escribir x;
    x <- x + 1;
Hasta x > 10;
```

Por contador (Para): cuando ya sabes de antemano cuántas repeticiones necesitas. El propio bucle se encarga de crear, incrementar (o decrementar, con `Con Paso`) y comprobar el contador — tú no debes tocarlo dentro del bucle.

```
Para x <- 1 Hasta 10 Hacer
    Escribir x;
FinPara
```

❓ **¡No Hay Preguntas Tontas! ¿`Repetir...Hasta` y `Mientras` no son lo mismo pero al revés?** Casi — la diferencia real está en **cuándo se pregunta**. `Mientras` pregunta primero y actúa después (puede que no actúe nunca). `Repetir...Hasta` actúa primero y pregunta después (actúa siempre al menos una vez). Para pasar de uno a otro, además de mover la condición, tienes que invertirla. Y ojo: no todos los lenguajes de programación tienen las tres formas — tendrás que saber traducir entre ellas.

14. Traza de un algoritmo

Trazar un algoritmo es ejecutarlo a mano, apuntando en una tabla cómo cambia el valor de cada variable en cada paso. Es la herramienta más básica (y más infravalorada) para depurar: antes de sospechar del ordenador, sospecha de tu lógica, y la traza te lo demuestra sobre el papel.

> ⚠ **Tu programa no hace lo que piensas…**
>
> …sino lo que **escribes**. La traza es la forma de comprobar, línea a línea, si de verdad coinciden.

15. Diagramas de flujo

Otra forma de representar un algoritmo, en lugar de con texto, es dibujarlo. Un **diagrama de flujo** (también llamado *ordinograma*) usa unos símbolos estándar unidos por flechas que marcan el siguiente paso:

- **Óvalo**: inicio o fin del algoritmo.
- **Paralelogramo**: entrada o salida de datos (leer/escribir).
- **Rectángulo**: una operación de proceso (una asignación, un cálculo).
- **Rombo**: una decisión — la ejecución se bifurca según sea verdadera o falsa.

Son muy útiles para ver de un vistazo algoritmos cortos, pero con muchas instrucciones se vuelven difíciles de seguir — por eso, a partir de cierto tamaño, se prefiere el pseudocódigo.

16. Subalgoritmos y programación modular

Imagina una tarea de 20 instrucciones que necesitas repetir en 7 lugares distintos del programa. Copiar y pegar esas 20 líneas siete veces funciona, pero es una mala idea: el programa es más largo, más difícil de leer, y si dentro de dos meses encuentras un fallo tienes que corregirlo en los 7 sitios.

La solución es un **subalgoritmo** (lo que en Java llamaremos **método**, unidades 2 y 4): un bloque de código con nombre propio al que puedes **llamar** desde donde lo necesites. A los datos que le pasas se les llama **parámetros**, y las variables que declara solo para su propio uso son sus **variables locales** (no existen fuera de él).

Hay dos tipos:

- **Función**: además de ejecutar sus instrucciones, **devuelve un valor** — se puede usar dentro de una expresión.

```
SubAlgoritmo resultado <- abs(x)
    Definir resultado Como Real;
    Si x < 0 Entonces
        resultado <- -x;
    Sino
        resultado <- x;
    FinSi
FinSubAlgoritmo
```

- **Procedimiento**: ejecuta sus instrucciones pero **no devuelve nada**.

```
SubAlgoritmo pulsacion(mensaje)
    Escribir mensaje, " Pulse tecla para continuar.";
    Esperar Tecla;
FinSubAlgoritmo
```

> **Por valor vs. por referencia**
>
> Por defecto, los parámetros que le pasas a un subalgoritmo son una **copia** — si el subalgoritmo los modifica, el dato original no cambia (paso **por valor**). Si en cambio se indica explícitamente **por referencia**, los cambios sí afectan al dato original. Volveremos sobre esta distinción, con mucho más detalle, cuando lleguemos a los métodos de Java.

Dividir un problema grande en subalgoritmos pequeños es la idea central de la **programación modular**: cada pieza se entiende, se prueba y se reutiliza por separado, en lugar de escribir un único bloque gigante de instrucciones. Además, los subalgoritmos más generales se pueden agrupar en **librerías** que reutilizas en otros programas — exactamente lo que hace Java con su librería estándar.

> **El error más habitual al empezar**
>
> Cuando alguien empieza a programar, la tentación es escribirlo todo en un único bloque enorme. Aunque “funcione”, es mucho más difícil de leer, probar y corregir. Acostúmbrate desde ya a pensar en subalgoritmos pequeños, cada uno con una única responsabilidad clara.

17. Errores de programación

No todos los errores son iguales, y conviene distinguirlos porque cada uno se detecta (y se corrige) de forma distinta:

- **Errores sintácticos**: el código no respeta la gramática del lenguaje (falta un paréntesis, sobra un punto y coma). Se detectan al **compilar**, antes de ejecutar nada.
- **Errores semánticos o lógicos**: el código compila y se ejecuta, pero **no hace lo que debería** — el algoritmo en sí está mal pensado. Son los más difíciles de detectar, porque no hay ningún aviso: solo un resultado incorrecto.
- **Errores en tiempo de ejecución**: el código es correcto en general, pero falla ante ciertos datos concretos — el ejemplo clásico es una división entre 0 (`velocidad = km / horas`, con `horas` valiendo 0). Solo aparecen si el programa llega a ejecutar esa situación concreta, lo que los hace fáciles de dejar pasar si no pruebas suficientes casos.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.	a) estructura de un programa · c) entornos integrados de desarrollo · i) comentarios en el código

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras